FINAL PROJECT

Graph-Based Smart Contract Auditing

A graph attention network prototype, tested by leave-one-contract-out and on new contracts

[Student Name] · Student ID [Student ID] · October 2026

MOTIVATION

Deployed contracts cannot be patched

- Smart contracts hold real assets on Ethereum; a flaw is exploitable as soon as it is deployed.
- Three classes studied: **Reentrancy**, **Timestamp dependency**, **Arithmetic error**.
- Existing tools disagree with each other and flag many real contracts (Durieux et al., 2020).
- Idea: treat a contract as a graph of control flow and data flow, and learn over it.

RESEARCH QUESTIONS

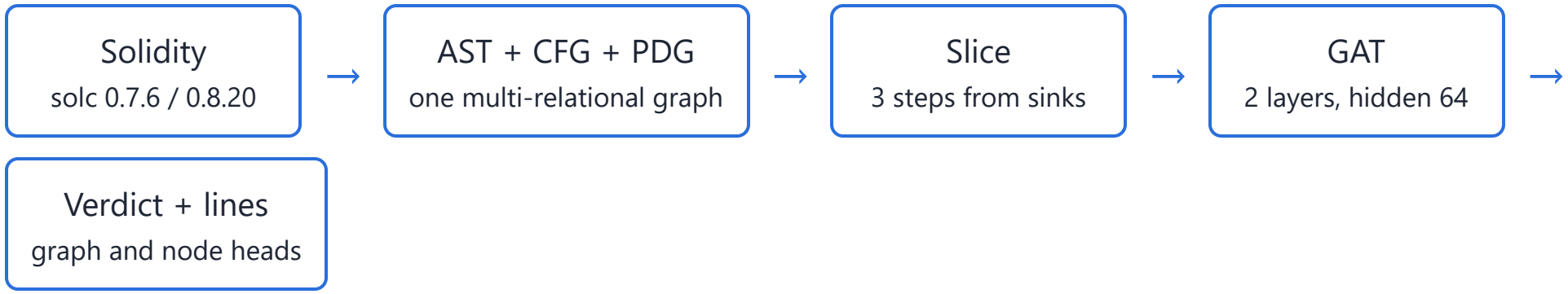
What this project asks

- **RQ1.** Can a GAT over a sliced contract graph separate vulnerable from safe contracts?
- **RQ2.** Does it generalize to contracts it has not seen, compared with simple baselines?
- **RQ3.** Can its node scores point to the vulnerable line?

> We report a feasibility result, not a detection rate.

PIPELINE

From source to verdict



Sinks: external calls, `block.timestamp`, arithmetic (binary and compound `+= -=`). Thresholds: 0.5 contract, 0.3 node (set by hand).

TWO PROTOCOLS

Held out, and new contracts

Leave-one-contract-out

- Six demo contracts; hold one out, train on five.
- 18 decisions per method, 3 positive.
- GAT over 5 seeds; baselines: sink-presence rule, logistic regression.

External test

- Train on all six; score six new contracts (different style; one solc 0.7.6).
- 18 decisions, 3 positive.
- Caveat: one contract informed the arithmetic fix, so this is not independent.

RESULTS: HELD OUT

Leave-one-contract-out, 18 decisions

Method	Precision	Recall	F1	Accuracy
GAT (5 seeds, identical)	0.000	0.000	0.000	0.778
Sink-presence rule	0.333	1.000	0.500	0.667
Logistic regression	0.000	0.000	0.000	0.833

> The GAT flags none of the three held-out vulnerable contracts, in any seed.

RESULTS: NEW CONTRACTS

External test, after the arithmetic fix

Method	Precision	Recall	F1	Accuracy	Recall before fix
GAT	1.000	1.000	1.000	1.000	0.667
Sink-presence rule	0.300	1.000	0.462	0.611	0.667
Logistic regression	0.000	0.000	0.000	0.833	0.000

> The fix was made after seeing LedgerVulnerable.sol, so this result is optimistic. It is not evidence of generalization.

IN-SAMPLE VS. HELD OUT

The fit does not transfer

18 / 18

correct when trained and tested on the same six contracts (F1 1.000)

0 / 3

held-out vulnerable contracts detected under leave-one-contract-out (recall 0.000)

> With one positive per class, holding out a contract removes the only example of its pattern. The fix also weakened separation on the demo set (Reentrancy 58%, safe Timestamp up to 29%).

FIGURE

Method comparison, held out



ERROR ANALYSIS

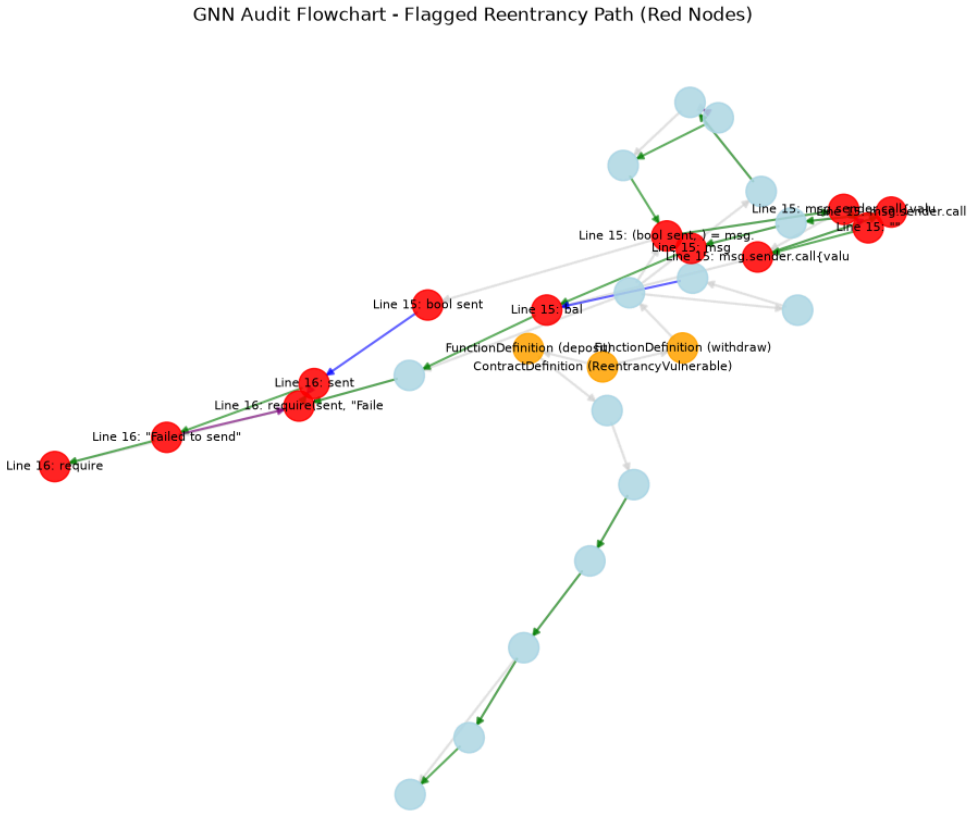
Why each error happens

- **GAT:** errors repeat in every seed. Held-out Reentrancy: the only training example is the safe contract, which scores the pattern as safe. A held-out safe contract scores 0.96 for Reentrancy.
- **Sink rule, keywords:** `send` matches inside `msg.sender` and `"Failed to send"`, so Arithmetic contracts are flagged for Reentrancy.
- **Sink rule, after the fix:** any `+=`, `-=` or `*` is an arithmetic sink, so six false positives.

LOCALIZATION

Lines: in-sample yes, held out no

- In-sample, Reentrancy is localized to lines 15 and 16 (external call and its check).
- In-sample, Timestamp is localized to line 9, after a label fix.
- Arithmetic is flagged at 99.88% but not localized.
- Held out: localization fails in all 15 vulnerable-contract evaluations.
- External: vault exact (14–15); raffle 13 plus one extra line; ledger not localized.



DEFECTS FOUND AND FIXED

Found while checking the work

- **Wrong training labels** for two contracts (lines 10 and 14, instead of 9 and 12).
- **Legacy-AST operator lookup** failed for solc 0.7.6, so the arithmetic contract had no sink.
- **Compound assignments ignored** by the arithmetic check. Found on the new contract; fixed; it weakened demo-set separation.
- **Unverifiable references** removed; the rest checked against publisher or arXiv pages.

> Pre-fix logs are kept in `results/logs/before_fixes/` and `results/logs/before_arithmetic_fix/`. Full list: `FAILURE_LOG.md`.

LIMITATIONS

What this does not show

- Six training and six external contracts, all written by the author: not real code.
- The external test informed a fix, so it is optimistic.
- Keyword features match substrings (`now`, `send`).
- Thresholds set by hand; compiler versions 0.4–0.6 untested.
- Public benchmarks (SmartBugs, SolidiFI) not yet used.

NEXT STEPS

What would make a detection claim possible

1. A fresh test set, written or chosen before any further change, and not used for design.
2. Evaluate on SolidiFI and SmartBugs with a held-out split.
3. Report precision, recall and F1 with confidence intervals.
4. Compare with Oyente, SmartCheck and Slither on the same contracts.

EVIDENCE

Everything is in the repository

- Code, saved weights, four executed notebooks, all metrics and logs.
- `results/metrics.json` and `results/external_test/metrics.json` hold the numbers in the manuscript and slides.
- Data card, model card, ethics, AI-use disclosure, failure log, reproducibility checklist.
- Reproduce: `python evaluate.py` and `python evaluate_external.py` (see `README.md`).